Guided Project: Movie Booking System Backend (MongoDB + Express + Mongoose)

Project Scope

Develop a **production-oriented backend system** for a movie ticket booking application similar to BookMyShow.

The system must support **two primary actors**:

1. Customer (User)

- Registers and authenticates
- Searches and explores movies
- Views shows and seat availability
- Books and cancels tickets
- Verifies identity using OTP

2. Admin (System Controller)

- Manages movies and show schedules
- Controls seat configurations and availability
- Monitors bookings and platform activity
- Enforces system-level rules and data integrity

The system must:

- Use **MongoDB with Mongoose**
- Follow **Express.js architecture**
- Implement **authentication, authorization, validation, and optimization**
- Ensure **data consistency, performance, and security**

Project Requirements

1. Database Connection & Project Initialization

Requirements

- Initialize project using npm
- Configure Express server
- Connect MongoDB using Mongoose
- Maintain environment variables

Actor Context

- **System-level setup** (no direct user interaction)

- Supports both **Admin and Customer workflows**

2. Schema Design & Data Modeling

Requirements

Design schemas for:

- User (Admin + Customer)
- Movie (Admin-managed)
- Show (Admin-managed)
- Booking (User-generated, Admin-visible)
- OTP (User authentication flow)

User Role Integration

- User schema must include:
  - role → admin | user
- Default:
  - user
- Admin:
  - pre-created or manually assigned

Relationships

- Booking → belongs to **User**
- Booking → belongs to **Show**
- Show → belongs to **Movie (Admin-controlled)**

Actor Responsibility Mapping

Entity	Created By	Accessed By
Movie	Admin	User + Admin
Show	Admin	User + Admin
Booking	User	User (own) + Admin (all)
OTP	System	User

This section should demonstrate:

- Schema design for multi-role systems
- Data ownership and access control

3. CRUD Operations with Role Awareness

Requirements

Admin Operations

- Create, update, delete:
  - Movies
  - Shows

User Operations

- Create:
  - Bookings
- Read:
  - Movies
  - Shows
  - Own bookings

Admin Read Access

- View:
  - All bookings
  - All users (optional)

This section should demonstrate:

- Controlled data modification
- Separation of responsibilities

4. Querying, Filtering & Pagination

Requirements

User Capabilities

- Search movies:
  - title (regex)
- Filter:
  - genre
  - rating range
- Paginate results

Admin Capabilities

- Filter bookings:
  - by movie
  - by show
  - by date

This section should demonstrate:

- Efficient querying for different user roles
- Handling large datasets

5. Aggregation Framework (Admin Insights)

Requirements

Admin Use Cases

- View:
    - most booked shows
    - top-rated movies
    - booking statistics

This section should demonstrate:

- Aggregation for analytics
- Business insights generation

6. Indexing & Performance Optimization

Requirements

- Apply indexes on:
    - movie title
    - user email
    - frequently filtered fields

Actor Context

- Improves:
    - User search experience
    - Admin reporting queries

7. Authentication System (JWT-Based)

Requirements

User & Admin

- Register (user only)
- Login (both)
- Receive JWT

Access Control

- Middleware must:
    - Authenticate (JWT)
    - Identify role

This section should demonstrate:

- Shared authentication layer
- Role differentiation after login

8. OTP-Based Verification System

Requirements

User Flow

- OTP generated during:
    - signup
    - password reset
- OTP validated before access

System Responsibilities

- Generate secure OTP
- Store hashed OTP
- Expire OTP using TTL

Admin Context

- Indirect role:
    - Monitor abuse via logs
    - Enforce stricter limits (future-ready)

9. Show & Seat Management

Requirements

Admin Responsibilities

- Create shows
- Define:
    - time
    - total seats
    - initial availability

User Capabilities

- View:
    - show timings
    - seat availability

This section should demonstrate:

- Admin-controlled system configuration
- User-facing data consumption

10. Booking System with Data Integrity

Requirements

User Actions

- Book tickets
- Cancel bookings

System Validations

- Prevent double booking

- Validate seat availability

Admin Oversight

- View all bookings
- Analyze booking patterns

This section should demonstrate:

- Multi-actor data flow
- System-enforced consistency

11. File Upload Handling (Admin Feature)

Requirements

Admin Only

- Upload movie posters

Constraints

- File type validation
- Size restriction
- Store file reference

12. Middleware Architecture

Requirements

Implement:

- Authentication middleware → identifies user
- Authorization middleware → checks role
- Logging middleware → tracks actions
- Validation middleware → ensures data correctness

Actor Context

- Applies to:
    - User requests
    - Admin operations

13. Error Handling System

Requirements

Handle:

- Invalid input
- Unauthorized access (User/Admin not logged in)

- Forbidden actions (wrong role)

Actor-Based Errors

- User accessing admin route → 403
- Unauthenticated access → 401

14. Rate Limiting & API Protection

Requirements

- Apply limits on:
    - login
    - OTP
    - booking APIs

Actor-Based Strategy

- Users → stricter limits
- Admin → controlled but less restrictive

15. Date, Time & TTL Handling

Requirements

- Store timestamps:
    - bookings
    - shows
- Expire:
    - OTP

Actor Context

- Users → booking history
- Admin → analytics & reports

16. Virtual Fields & Derived Data

Requirements

- Add computed fields:
    - Movie → release status
    - Booking → derived summary

Actor Context

- Improves:
    - User UI display

    - Admin reporting

17. Logging & Monitoring

Requirements

- Log:
    - user actions
    - admin actions
    - system errors

Admin Use Case

- Monitor:
    - failures
    - suspicious activity

18. API Design & Structure

Requirements

Structure project into:

- routes
- controllers
- models
- middleware
- utils

Actor Context

- Separate:
    - user routes
    - admin routes

19. Validation & Edge Case Handling

Requirements

Handle:

- Invalid inputs
- Unauthorized role access
- Seat conflicts
- Expired OTP

Actor Context

- Prevent misuse by:
    - users
    - incorrect admin actions

Suggested Functional Flow

1. Admin creates movies and shows
2. User registers and verifies via OTP
3. User logs in and receives JWT
4. User browses and filters movies
5. User views shows and selects seats
6. User books tickets
7. System validates and confirms booking
8. User can cancel booking
9. Admin monitors bookings and updates system

1. FOLDER STRUCTURE

```
movie-booking-backend/
|
 ├── src/
|   ├── config/
|   |   ├── db.js
|   |   └── env.js
|   |
|   ├── models/
|   |   ├── User.js
|   |   ├── Movie.js
|   |   ├── Show.js
|   |   ├── Booking.js
|   |   └── OTP.js
|   |
|   ├── routes/
|   |   ├── auth.routes.js
|   |   ├── user.routes.js
|   |   ├── admin.routes.js
|   |   ├── movie.routes.js
|   |   ├── show.routes.js
|   |   └── booking.routes.js
|   |
|   ├── controllers/
|   |   ├── auth.controller.js
|   |   ├── user.controller.js
|   |   ├── admin.controller.js
|   |   ├── movie.controller.js
|   |   ├── show.controller.js
|   |   └── booking.controller.js
|   |
|   ├── services/         (business logic layer)
|   |   ├── auth.service.js
|   |   ├── booking.service.js
|   |   ├── otp.service.js
|   |   └── movie.service.js
|   |
|   ├── middleware/
|   |   ├── auth.middleware.js
|   |   ├── role.middleware.js
|   |   ├── validation.middleware.js
|   |   ├── error.middleware.js
|   |   └── rateLimit.middleware.js
|   |
|   ├── utils/
|   |   ├── logger.js
|   |   ├── customError.js
|   |   └── helpers.js
|   |
```

```
|   ├── uploads/          (posters storage)
|   |
|   ├── app.js
|   └── server.js
|
├── package.json
└── .env
```

- **Routes → only define endpoints**
- **Controllers → handle request/response**
- **Services → contain business logic**
- **Models → DB layer**
- **Middleware → cross-cutting concerns**

This separation ensures:

- Scalability
- Testability
- Clean codebase

2. API DESIGN (ROLE-WISE)

We divide APIs into:

1. Public APIs (No Auth Required)

2. User APIs (Authenticated User)

3. Admin APIs (Admin Only)

2.1 AUTH APIs
1. Register User
Endpoint
POST /api/auth/register
Access
- Public (User)

Request
```
{
  "name": "Rakesh",
  "email": "rakesh@email.com",
  "password": "Password@123"
}
```
Response
```
{
  "success": true,
  "message": "User registered successfully. OTP sent.",
  "data": {
```

```
    "email": "rakesh@email.com"
  }
}
```


2. Verify OTP

POST /api/auth/verify-otp

Request


```
{
  "email": "rakesh@email.com",
  "otp": "123456"
}
```


Response


```
{
  "success": true,
  "message": "OTP verified successfully"
}
```


3. Login

POST /api/auth/login

Request


```
{
  "email": "rakesh@email.com",
  "password": "Password@123"
}
```


Response


```
{
  "success": true,
  "token": "JWT_TOKEN",
  "data": {
    "userId": "123",
    "role": "user"
  }
}
```


2.2 MOVIE APIs

1. Get Movies (User + Public)

GET /api/movies

Query Params

?genre=action&rating=4&page=1&limit=5&sort=rating

Response


```
{
  "success": true,
  "data": {
    "movies": [],
    "pagination": {
      "page": 1,
      "limit": 5,
      "total": 50
    }
  }
}
```

2. Create Movie (Admin Only)
POST /api/movies
Headers
Authorization: Bearer TOKEN
Request
```
{
  "title": "Inception",
  "genre": "Sci-Fi",
  "rating": 4.8,
  "duration": 120,
  "releaseDate": "2024-01-01"
}
```
Response
```
{
  "success": true,
  "message": "Movie created successfully"
}
```
3. Update Movie (Admin)
PUT /api/movies/:id
4. Delete Movie (Admin)
DELETE /api/movies/:id
2.3 SHOW APIs
1. Create Show (Admin)
POST /api/shows
Request
```
{
  "movieId": "movie123",
  "date": "2026-05-01",
  "time": "18:00",
  "totalSeats": 100
}
```
2. Get Shows (User)
GET /api/shows?movieId=123
Response
```
{
  "success": true,
  "data": [
    {
      "showId": "1",
      "time": "18:00",
      "availableSeats": 50
    }
  ]
}
```
2.4 BOOKING APIs
1. Create Booking (User)
POST /api/bookings
Request

```
{
  "showId": "show123",
  "seats": ["A1", "A2"]
}
```

Response

```
{
  "success": true,
  "message": "Booking confirmed",
  "data": {
    "bookingId": "b123",
    "seats": ["A1", "A2"]
  }
}
```

2. Get My Bookings (User)

GET /api/bookings/my

3. Cancel Booking (User)

DELETE /api/bookings/:id

4. Get All Bookings (Admin)

GET /api/bookings

Query

?movieId=123&date=2026-05-01

2.5 FILE UPLOAD API (ADMIN)

POST /api/upload/poster

Type

- multipart/form-data

Response

```
{
  "success": true,
  "url": "/uploads/movie1.jpg"
}
```

3. STANDARD RESPONSE STRUCTURE

Every API must follow a consistent structure:

Success

```
{
  "success": true,
  "message": "Optional message",
  "data": {}
}
```

Error

```
{
  "success": false,
  "message": "Error message",
  "error": {
    "code": "INVALID_INPUT",
    "details": "Rating must be between 1 and 5"
  }
}
```

4. ROLE-BASED ACCESS SUMMARY

API	User	Admin
Register/Login	Yes	Yes
View Movies	Yes	Yes
Create Movie	No	Yes
Create Show	No	Yes
Book Tickets	Yes	No
View Own Booking	Yes	No
View All Bookings	No	Yes

5. CRITICAL IMPLEMENTATION INSIGHTS

1. Middleware Flow

Request → Auth Middleware → Role Middleware → Controller → Service → Response

2. Booking Flow (Important)

User → Select Seats → Validate Seats → Lock → Save Booking → Update Show

3. OTP Flow

Register → Generate OTP → Store (TTL) → Verify → Activate User